UNIVERSITY MOHAMED BOUDIAF OF M'SILA

Faculty of Mathematics and Computer Science

Department of Computer Science

TP N°2 — Laboratory Report

Data Manipulation with R

FDTA — Fouille de Données, Techniques et Applications

Prepared by: Younes Barkat

Level: Master 01 — Artificial Intelligence

Supervised by: Dr. Taher MEHANNI

Academic Year: 2025 / 2026

TP N°2: Data Manipulation with R

Objectives

This lab develops practical R skills across five core areas: installing and loading external packages, manipulating data frames with indexing and filtering, applying descriptive statistics and cross-tabulations, producing and comparing graphical visualisations, and exporting enriched datasets to Excel.

Step 1: Clearing the Memory Environment

```
rm(list=ls())
```

```
> rm(list=ls())
```

rm(list=ls()) removes every object currently held in the R workspace, ensuring no residual variables from a previous session interfere with the new analysis. The Environment panel confirms the clean slate by displaying 'Environment is empty'.

Step 2: Installing the xlsx Package

```
install.packages("xlsx")
```

```
> install.packages("xlsx")
WARNING: Rtools is required to build R packages but is not currently installed. Please download and install th
e appropriate version of Rtools before proceeding:

https://cran.rstudio.com/bin/windows/Rtools/

Installing package into 'C:/Users/Younes/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)

also installing the dependency 'xlsxjars'

trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/xlsxjars_0.9.0.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/xlsx_0.6.5.zip'
package 'xlsxjars' successfully unpacked and MD5 sums checked
package 'xlsx' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
        C:\Users\Younes\AppData\Local\Temp\RtmpkjRm4N\downloaded_packages
```

install.packages() fetches the xlsx package and its Java-based dependency xlsxjars from the CRAN mirror. Both archives are unpacked and MD5-verified successfully, making Excel I/O functions available for this session. A note about Rtools is displayed but does not prevent the binary installation from completing.

Step 3: Loading the xlsx Library

```
library(xlsx)
```

```
> library(xlsx)
```

library() attaches the xlsx namespace to the search path, exposing read.xlsx() and write.xlsx() for use. No error messages are returned, confirming that the package and its Java runtime dependency are correctly configured.

Step 4: Setting the Working Directory

```
setwd("C:/Users/Younes/Desktop/TP_FDDTA/TP2")
```

```
> setwd("C:/Users/Younes/Desktop/TP_FDDTA/TP2")
```

setwd() redirects R's file lookup to the TP2 project folder, so subsequent read/write calls can reference filenames without full absolute paths. This is a standard reproducibility practice that keeps all session artefacts co-located with the source data.

Step 5: Loading the Heart Dataset

```
heart.full <- read.xlsx(file="heart.xlsx", sheetIndex=1, header=T)
```

```
> heart.full <- read.xlsx(file="heart.xlsx",sheetIndex=1,header=T)
```

	age	sexe	typedouleur	sucre	tauxmax	angine	depression	coeur
1	70	masculin	D	A	109	non	24	presence
2	67	feminin	C	A	160	non	16	absence
3	57	masculin	B	A	141	non	3	presence
4	64	masculin	D	A	105	oui	2	absence
5	74	feminin	B	A	121	oui	2	absence
6	65	masculin	D	A	140	non	4	absence
7	56	masculin	C	B	142	oui	6	presence
8	59	masculin	D	A	142	oui	12	presence
9	60	masculin	D	A	170	non	12	presence
10	63	feminin	D	A	154	non	40	presence

read.xlsx() parses the first sheet of heart.xlsx into a data frame named heart.full, with column names taken from the header row. The viewer confirms 8 variables: age, sexe, typedouleur, sucre, tauxmax, angine, depression, and coeur — covering demographic and clinical attributes for 270 patient records.

Step 6: Summary Statistics of the Dataset

```
print(summary(heart.full))
```

```
> print(summary(heart.full))
      age            sexe           typedouleur          sucre            tauxmax
 Min.   :29.00   Length:270         Length:270         Length:270         Min.   : 71.0
 1st Qu.:48.00   Class :character   Class :character   Class :character   1st Qu.:133.0
 Median :55.00   Mode  :character   Mode  :character   Mode  :character   Median :153.5
 Mean   :54.43                                                           Mean   :149.7
 3rd Qu.:61.00                                                           3rd Qu.:166.0
 Max.   :77.00                                                           Max.   :202.0
    angine           depression      coeur
 Length:270         Min.   : 0.0   Length:270
 Class :character   1st Qu.: 0.0   Class :character
 Mode  :character   Median : 8.0   Mode  :character
                    Mean   :10.5
                    3rd Qu.:16.0
                    Max.   :62.0
```

summary() produces per-variable descriptive statistics: numeric columns (age, tauxmax, depression) show min, quartiles, mean and max, while character columns display class and mode only. Age ranges from 29 to 77 years with a mean of 54.43, and tauxmax spans 71 to 202 bpm.

Step 7: Listing Workspace Objects

```
ls()
```

```
> ls()
[1] "heart.full"
```

ls() enumerates all objects currently loaded in the Global Environment. Only heart.full is present, verifying that no unintended side-effects from package loading have polluted the workspace.

Step 8: Accessing the Age Variable

```
print(heart.full$age)
```

```
> print(heart.full$age)
  [1] 70 67 57 64 74 65 56 59 60 63 59 53 44 61 57 71 46 53 64 40 67 48 43 47 54 48 46 51 58 71 57 66 37
 [34] 59 50 48 61 59 42 48 40 62 44 46 59 58 49 44 66 65 42 52 65 63 45 41 61 60 59 62 57 51 44 60 63 57
 [67] 51 58 44 47 61 57 70 76 67 45 45 39 42 56 58 35 58 41 57 42 62 59 41 50 59 61 54 54 52 47 66 58 64
[100] 50 44 67 49 57 63 48 51 60 59 45 55 41 60 54 42 49 46 56 66 56 49 54 57 65 54 54 62 52 52 60 63 66
[133] 42 64 54 46 67 56 34 57 64 59 50 51 54 53 52 40 58 41 41 50 54 64 51 46 55 45 56 66 38 62 55 58 43
[166] 64 50 53 45 65 69 69 67 68 34 62 51 46 67 50 42 56 41 42 53 43 56 52 62 70 54 70 54 35 48 55 58 54
[199] 69 77 68 58 60 51 55 52 60 58 64 37 59 51 43 58 29 41 63 51 54 44 54 65 57 63 35 41 62 43 58 52 61
[232] 39 45 52 62 62 53 43 47 52 68 39 53 62 51 60 65 65 60 60 54 44 44 51 59 71 61 55 64 43 58 60 58 49
[265] 48 52 44 56 57 67
```

The $ operator extracts the age column as a numeric vector of 270 values. R prints them in labelled rows of 9, making it easy to spot the distribution range — values visually cluster in the 40–70 bracket.

Step 9: Data Type of the Age Column

```
class(heart.full$age)
```

```
> class(heart.full$age)
[1] "numeric"
```

class() returns 'numeric', confirming age is stored as a continuous double-precision variable. This type is required for arithmetic operations such as mean, quantile, and tapply aggregations performed in later steps.

Step 10: Length of the Age Vector

```
length(heart.full$age)
```

```
> length(heart.full$age)
[1] 270
```

length() reports 270 observations, matching the expected dataset size. This confirms no rows were dropped during import and that the full patient cohort is available for analysis.

Step 11: Accessing Age by Index Range

```
heart.full$age[1:10]
```

```
> heart.full$age[1:10]
 [1] 70 67 57 64 74 65 56 59 60 63
```

Bracket notation with a colon range selects the first 10 age values: 70 67 57 64 74 65 56 59 60 63. This slice confirms correct row ordering and gives a quick preview of the age distribution at the dataset's start.

Step 12: Accessing Scattered Age Values

```
heart.full$age[c(2,5,8)]
```

```
> heart.full$age[c(2,5,8)]
[1] 67 74 59
```

c() creates an index vector, allowing non-contiguous row selection. Rows 2, 5, and 8 return ages 67, 74, and 59 respectively, demonstrating flexible element access without subsetting the entire data frame.

Step 13: Mean Age

```
mean(heart.full$age)
```

```
> mean(heart.full$age)
[1] 54.43333
```

The mean age across all 270 patients is 54.43 years. This central tendency value will later serve as a baseline when comparing sub-group means broken down by sex and angina status using tapply().

Step 14: Quantiles of Age

```
quantile(heart.full$age, probs=c(0.1, 0.5, 0.9))
```

```
> quantile(heart.full$age,probs=c(0.1,0.5,0.9))
10% 50% 90%
 42  55  66
```

The 10th percentile is 42, the median 55, and the 90th percentile 66. The relatively narrow spread between the 10th and 90th percentiles (24 years) indicates a moderately concentrated age distribution skewed slightly toward middle-aged adults.

Step 15: Mean Age for First 10 Observations

```
mean(heart.full$age[1:10])
```

```
> mean(heart.full$age[1:10])
[1] 63.5
```

Restricting the mean calculation to the first 10 rows yields 63.5 years — notably higher than the overall mean of 54.43. This illustrates how partial-sample statistics can deviate significantly from the full-dataset estimate, underscoring the importance of using complete data.

Step 16: Data Type of the Sex Column

```
class(heart.full$sexe)
```

```
> class(heart.full$sexe)
[1] "character"
```

class() returns 'character' for the sexe column, meaning gender values are stored as plain strings rather than factors. Operations like table() still work correctly, but factor-specific functions such as levels() will return NULL as seen in the next step.

Step 17: Factor Levels of Sex

```
levels(heart.full$sexe)
```

```
> levels(heart.full$sexe)
NULL
```

levels() returns NULL because sexe is a character vector, not a factor. In R, factor levels are only defined when a column is explicitly cast with as.factor(). This distinction matters for ordered categorical analysis and model encoding.

Step 18: Length of the Sex Vector

```
length(heart.full$sexe)
```

```
> length(heart.full$sexe)
[1] 270
```

270 observations are confirmed for sexe, consistent with the dataset size established in Step 10. Every row has a gender entry, so no missing values affect the sex-based grouping analyses that follow.

Step 19: First 10 Values of Sex

```
heart.full$sexe[1:10]
```

```
> heart.full$sexe[1:10]
 [1] "masculin" "feminin"  "masculin" "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin"
[10] "feminin"
```

The first 10 gender entries are predominantly 'masculin', with 'feminin' appearing at positions 2, 5, and 10. This early preview is consistent with the dataset's overall sex imbalance (183 male vs. 87 female) confirmed in the frequency table in Step 21.

Step 20: Scattered Sex Values

```
heart.full$sexe[c(2,5,8)]
```

```
> heart.full$sexe[c(2,5,8)]
[1] "feminin"  "feminin"  "masculin"
```

Rows 2, 5, and 8 return 'feminin', 'feminin', 'masculin'. This targeted access demonstrates that non-contiguous character indexing works identically for string vectors as for numeric ones.

Step 21: Frequency Table for Sex

```
table(heart.full$sexe)
```

```
> table(heart.full$sexe)

 feminin masculin
      87       183
```

table() tallies 87 female and 183 male patients — a 32/68 split. This imbalance must be considered when interpreting sex-stratified statistics, as smaller female subgroup sizes produce less stable estimates.

Step 22: Internal Codes via unclass()

```
unclass(heart.full$sexe)
```

```
> unclass(heart.full$sexe)
  [1] "masculin" "feminin"  "masculin" "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin"
 [10] "feminin"  "masculin" "masculin" "masculin" "masculin" "feminin"  "feminin"  "masculin" "masculin"
 [19] "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"  "feminin"  "feminin"
 [28] "feminin"  "masculin" "feminin"  "masculin" "masculin" "feminin"  "masculin" "masculin" "masculin"
 [37] "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"  "masculin" "masculin" "masculin"
 [46] "masculin" "masculin" "masculin" "masculin" "feminin"  "masculin" "masculin" "feminin"  "feminin"
 [55] "feminin"  "feminin"  "masculin" "feminin"  "feminin"  "masculin" "masculin" "feminin"  "masculin"
 [64] "feminin"  "masculin" "masculin" "masculin" "feminin"  "feminin"  "masculin" "masculin" "feminin"
 [73] "masculin" "feminin"  "feminin"  "masculin" "masculin" "feminin"  "feminin"  "masculin" "masculin"
 [82] "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"  "masculin"
 [91] "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"
[100] "feminin"  "feminin"  "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin" "feminin"
[109] "masculin" "feminin"  "feminin"  "masculin" "feminin"  "feminin"  "masculin" "feminin"  "masculin"
[118] "feminin"  "feminin"  "masculin" "masculin" "masculin" "masculin" "feminin"  "masculin" "feminin"
[127] "masculin" "feminin"  "masculin" "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin"
[136] "feminin"  "feminin"  "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin" "masculin"
[145] "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"  "masculin" "feminin"  "feminin"
[154] "feminin"  "feminin"  "feminin"  "masculin" "masculin" "masculin" "feminin"  "masculin" "feminin"
[163] "masculin" "masculin" "masculin" "feminin"  "feminin"  "masculin" "feminin"  "masculin" "masculin"
[172] "masculin" "masculin" "feminin"  "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin"
[181] "masculin" "feminin"  "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin" "feminin"
[190] "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"  "feminin"  "feminin"
[199] "feminin"  "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"
[208] "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"  "masculin" "masculin" "feminin"
[217] "feminin"  "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"
[226] "masculin" "feminin"  "feminin"  "feminin"  "masculin" "feminin"  "masculin" "masculin" "masculin"
[235] "masculin" "feminin"  "feminin"  "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"
[244] "feminin"  "feminin"  "masculin" "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin"
[253] "masculin" "masculin" "masculin" "feminin"  "masculin" "masculin" "masculin" "masculin" "feminin"
[262] "masculin" "masculin" "masculin" "masculin" "masculin" "masculin" "feminin"  "masculin" "masculin"
```

unclass() on a character column simply returns the raw string values with a NULL attr, since no integer encoding exists for non-factors. The output spans all 270 rows, displaying the underlying character representation of each gender entry.

Step 23: Copying Dataset to Matrix m

```
m <- heart.full
```

```
> m <- heart.full
```

	age	sexe	typedouleur	sucre	tauxmax	angine	depression	coeur
1	70	masculin	D	A	109	non	24	presence
2	67	feminin	C	A	160	non	16	absence
3	57	masculin	B	A	141	non	3	presence
4	64	masculin	D	A	105	oui	2	absence
5	74	feminin	B	A	121	oui	2	absence
6	65	masculin	D	A	140	non	4	absence
7	56	masculin	C	B	142	oui	6	presence
8	59	masculin	D	A	142	oui	12	presence
9	60	masculin	D	A	170	non	12	presence
10	63	feminin	D	A	154	non	40	presence

Assigning heart.full to m creates a working copy, preserving the original object intact. Both variables point to identical data frames at this stage; subsequent modifications to m will not affect heart.full.

Step 24: Row and Column Counts

```
nrow(m)
ncol(m)
```

```
> nrow(m)
[1] 270
```

```
> ncol(m)
[1] 8
```

nrow() returns 270 (patients) and ncol() returns 8 (variables). These dimensions confirm the data frame's structure matches the original Excel file and serve as a sanity check before any matrix-style indexing.

Step 25: Single Element Access: m[1,1]

```
m[1,1]
```

```
> m[1,1]
[1] 70
```

m[1,1] extracts the value at row 1, column 1, returning 70 — the age of the first patient. This confirms that the data frame supports standard matrix-style [row, col] indexing, treating it as a two-dimensional structure.

Step 26: Sub-matrix: Rows 1–5, Columns 2–4

```
m[1:5, 2:4]
```

```
> m[1:5,2:4]
      sexe typedouleur sucre
1 masculin           D     A
2  feminin           C     A
3 masculin           B     A
4 masculin           D     A
5  feminin           B     A
```

Slicing rows 1–5 and columns 2–4 yields a 5×3 sub-frame showing sexe, typedouleur, and sucre. This demonstrates that range-based double indexing extracts rectangular blocks efficiently, which is useful for exploratory inspection of specific variable groups.

Step 27: Scattered Row and Column Access

```
m[c(2,5,8), 2:4]
```

```
> m[c(2,5,8),2:4]
      sexe typedouleur sucre
2  feminin           C     A
5  feminin           B     A
8 masculin           D     A
```

Combining a c() index for rows with a range for columns retrieves three non-consecutive records. All three rows (2, 5, 8) are female with pain types C, B, D and sugar type A, illustrating the power of mixed-index selection for spot-checking specific observations.

Step 28: Mixed Column Selection: m[2:4, c(1,3,6)]

```
m[2:4, c(1,3,6)]
```

```
> m[2:4,c(1,3,6)]
  age typedouleur angine
2  67           C    non
3  57           B    non
4  64           D    oui
```

Selecting rows 2–4 and non-adjacent columns 1, 3, and 6 returns age, typedouleur, and angine. Rows 2 and 3 show no angina; row 4 reports 'oui'. This mixed indexing style allows targeted variable extraction without dropping or reordering the data frame.

Step 29: Entire First Row

```
m[1,]
```

```
> m[1,]
  age     sexe typedouleur sucre tauxmax angine depression    coeur
1  70 masculin           D     A     109    non         24 presence
```

Omitting the column index retrieves all 8 variables for patient 1: age 70, masculin, type D pain, sugar A, tauxmax 109, no angina, depression 24, and coeur 'presence'. This full-row view is useful for individual record inspection.

Step 30: Named Column Access: m[5:6, c('age','angine')]

```
m[5:6, c("age","angine")]
```

```
> m[5:6,c("age","angine")]
  age angine
5  74    oui
6  65    non
```

Columns can be selected by name as well as by index. Rows 5 and 6 show ages 74 and 65 with angina 'oui' and 'non' respectively. Named indexing improves readability and guards against errors caused by column reordering.

Step 31: Boolean Filter: Age < 30

```
m[m$age<30,]
```

```
> m[m$age<30,]
    age     sexe typedouleur sucre tauxmax angine depression   coeur
215  29 masculin           B     A     202    non          0 absence
```

Logical indexing with a Boolean condition isolates only row 215, where age is 29. This is the youngest patient in the dataset — a 29-year-old male with type B pain and no heart disease presence — demonstrating threshold-based row filtering.

Step 32: Combined Filter: Age ≤ 34 AND Sex = Masculin

```
m[m$age<=34 & m$sexe=="masculin", c("age","sexe","coeur")]
```

```
> m[m$age<=34 & m$sexe=="masculin",c("age","sexe","coeur")]
    age     sexe   coeur
175  34 masculin absence
215  29 masculin absence
```

The & operator applies two conditions simultaneously, returning only rows 175 (age 34) and 215 (age 29) — both male with 'absence' of heart disease. Combined Boolean filtering enables multi-criteria cohort selection efficiently.

Step 33: OR Filter: Age ≤ 34 OR Age ≥ 76

```
m[m$age<=34 | m$age>=76, c("age","sexe")]
```

```
> m[m$age<=34 | m$age>=76,c("age","sexe")]
    age     sexe
74   76  feminin
139  34  feminin
175  34 masculin
200  77 masculin
215  29 masculin
```

The | operator selects extreme-age patients: five records appear — ages 34 (×2 female), 34 (male), 77 (male), and 29 (male). The OR condition is essential for isolating multiple non-contiguous value ranges without explicitly listing every qualifying value.

Step 34: Subset a: Males Aged ≤ 45, Columns angine & coeur

```
a <- m[m$age<=45 & m$sexe=="masculin", c("angine","coeur")]
```

```
> a <- m[m$age<=45 & m$sexe=="masculin",c("angine","coeur")]
```

	angine	coeur
13	non	absence
20	oui	absence
23	non	absence
39	non	absence
41	non	presence
43	oui	absence
48	non	presence
51	oui	presence
63	non	absence
76	oui	presence

Subset a captures young male patients, retaining only the angina and heart disease columns. The viewer shows 10 visible rows (of 39 total), with a mix of 'non'/'oui' angina and 'absence'/'presence' heart status — the raw material for the cross-tabulations in the next steps.

Step 35: Dimensions of Subset a

```
nrow(a)
ncol(a)
```

```
> nrow(a)
[1] 39
```

```
> ncol(a)
[1] 2
```

nrow(a) = 39 confirms 39 young male patients satisfy the combined age/sex filter. ncol(a) = 2 reflects the column restriction to angine and coeur. These dimensions guide interpretation of the frequency tables that follow.

Step 36: Frequency Table: Heart Disease in Subset a

```
table(a$coeur)
```

```
> table(a$coeur)

 absence presence
      27       12
```

Among the 39 young male patients, 27 show absence and 12 show presence of heart disease. This 69/31 split suggests that even in this younger male cohort, a meaningful proportion already exhibits cardiac pathology.

Step 37: Cross-tabulation k: Angina × Heart Disease

```
k <- table(a$angine, a$coeur)
class(k)
print(k)
print(k[1,2])
```

```
> k <- table(a$angine,a$coeur)
```

Values	
k	'table' int [1:2, 1:2] 24 3 5 7

```
> class(k)
[1] "table"
```

```
> print(k)

     absence presence
non       24        5
oui        3        7
```

```
> print(k[1,2])
[1] 5
```

k is a 2×2 contingency table of type 'table': non-angina patients split 24 absent / 5 present, while angina patients split 3 absent / 7 present. k[1,2] = 5 accesses the non-angina/presence cell. The higher presence rate among angina patients (7/10 vs 5/29) suggests angina is a meaningful predictor.

Step 38: Cross-tabulation e: Angina × Heart Disease (Full Dataset)

```
e <- table(m$angine, m$coeur)
print(e)
```

```
> e <- table(m$angine,m$coeur)
```

Values	
e	'table' int [1:2, 1:2] 127 23 54 66

```
> print(e)

     absence presence
non      127       54
oui       23       66
```

Across all 270 patients, the table shows: non-angina 127 absent / 54 present; angina 23 absent / 66 present. The angina group has a much higher disease rate (66/89 ≈ 74%), strongly associating angina with cardiac pathology at the population level.

Step 39: Total Angina Patients

```
print(sum(e[2,]))
```

```
> print(sum(e[2,]))
[1] 89
```

sum(e[2,]) sums the entire second row (angina = 'oui'): 23 + 66 = 89 patients with angina. This marginal total is the denominator for the disease proportion computed in the next step.

Step 40: Proportion of Diseased Among Angina Patients

```
print(e[2,2]/sum(e[2,]))
```

```
> print(e[2,2]/sum(e[2,]))
[1] 0.741573
```

74.16% of patients with angina have confirmed heart disease. This high proportion reinforces angina as a clinically significant symptom, and the result is directly computed from the contingency table without any additional library.

Step 41: Mean Age by Sex

```
tapply(X=m$age, INDEX=m$sexe, mean)
```

```
> tapply(X=m$age,INDEX=m$sexe,mean)
 feminin masculin
55.67816 53.84153
```

tapply() applies mean() separately to each sex group. Female patients average 55.68 years and male patients 53.84 years — a difference of under 2 years, indicating comparable age distributions between the two groups in this cohort.

Step 42: Mean Age by Sex and Angina (2D tapply)

```
b <- tapply(X=m$age, INDEX=list(m$sexe, m$angine), mean)
print(b)
```

```
> b <-tapply(X=m$age,INDEX=list(m$sexe,m$angine),mean)
```

	non	oui
feminin	55.72464	55.50000
masculin	52.62500	55.76056

```
> print(b)
              non      oui
feminin  55.72464 55.50000
masculin 52.62500 55.76056
```

A two-dimensional tapply produces a 2×2 matrix: female/non-angina 55.72, female/angina 55.50, male/non-angina 52.63, male/angina 55.76. The highest mean age appears in male angina patients, suggesting age may interact with sex in angina onset patterns.

Step 43: Range Between Min and Max Sub-group Means

```
d <- max(b)-min(b)
print(d)
```

```
> d <- max(b)-min(b)
```

Values	
d	3.13556338028169

```
> print(d)
[1] 3.135563
```

The gap between the largest (55.76) and smallest (52.63) sub-group mean age is 3.14 years. This relatively modest range suggests that age alone does not create dramatic differences across the sex–angina sub-groups, and other clinical variables may be more discriminating.

Step 44: Age Range by Sex

```
tapply(X=m$age, INDEX=m$sexe, function(x){max(x)-min(x)})
```

```
> tapply(X=m$age,INDEX=m$sexe,function(x){max(x)-min(x)})
 feminin masculin
      42       48
```

Female patients span 42 years (min to max) and male patients 48 years. The anonymous function max(x)−min(x) is passed directly to tapply, demonstrating how arbitrary custom aggregation functions can replace built-in ones in grouped computations.

Step 45: List Workspace Objects

```
ls()
```

```
> ls()
[1] "a"          "b"          "d"          "e"          "heart.full" "k"          "m"
```

At this point the workspace contains a, b, d, e, heart.full, k, and m. Each object corresponds to a step-specific computation no garbage variables have accumulated, confirming disciplined use of named assignments throughout the session.

Step 46: First 6 Values of Age (head)

```
head(m$age)
```

```
> head(m$age)
[1] 70 67 57 64 74 65
```

head() retrieves the six leading age values: 70 67 57 64 74 65. This preview is the baseline before applying sort(), allowing a direct before/after comparison of the ordering effect in the next step.

Step 47: Sorting the Age Vector

```
age2 <- sort(m$age)
head(age2)
```

```
> age2 <- sort(m$age)
```

Values	
age2	num [1:270] 29 34 34 35 35 35 37 37 38…

```
> head(age2)
[1] 29 34 34 35 35 35
```

sort() returns a new ascending vector age2 stored in the environment. head(age2) confirms the first six values are 29 34 34 35 35 35 — the youngest patients — validating the sort operation.

Step 48: Sorting the Data Frame by Age

```
head(order(m$age))
head(m[order(m$age),])
```

```
> head(order(m$age))
[1] 215 139 175  82 194 225
```

```
> head(m[order(m$age),])
    age     sexe typedouleur sucre tauxmax angine depression    coeur
215  29 masculin           B     A     202    non          0  absence
139  34  feminin           B     A     192    non          7  absence
175  34 masculin           A     A     174    non          0  absence
82   35 masculin           D     A     130    oui         16 presence
194  35 masculin           D     A     156    oui          0 presence
225  35  feminin           D     A     182    non         14  absence
```

order() returns the row indices that would sort the data frame — [215 139 175 82 194 225] — meaning row 215 holds the youngest patient. Applying these indices to m reorders all columns simultaneously, producing a fully sorted data frame without altering the original.

Step 49: Sorting by Two Criteria: Age then tauxmax

```
head(m[order(m$age, m$tauxmax),])
```

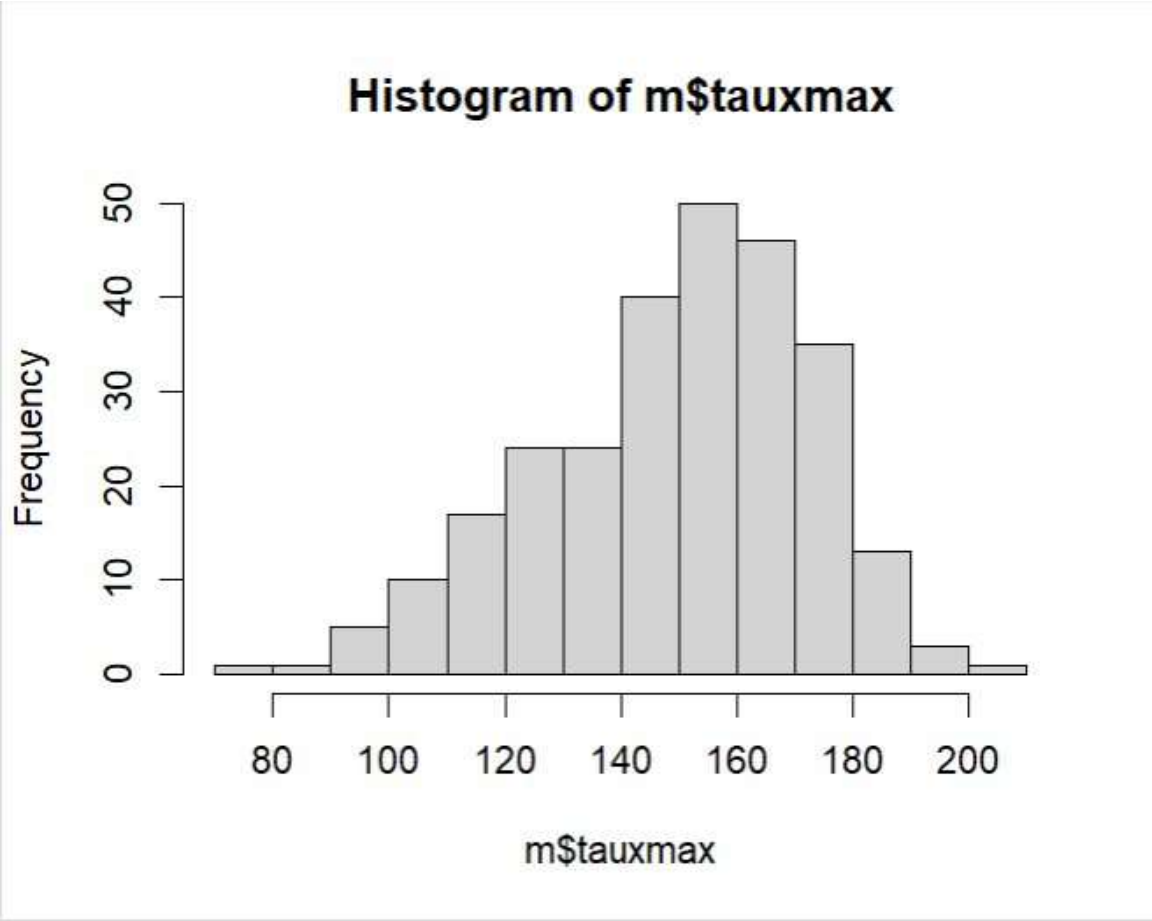
```
> head(m[order(m$age,m$tauxmax),])
    age     sexe typedouleur sucre tauxmax angine depression    coeur
215  29 masculin           B     A     202    non          0  absence
175  34 masculin           A     A     174    non          0  absence
139  34  feminin           B     A     192    non          7  absence
82   35 masculin           D     A     130    oui         16 presence
194  35 masculin           D     A     156    oui          0 presence
225  35  feminin           D     A     182    non         14  absence
```

Passing two variables to order() creates a hierarchical sort: age is the primary key and tauxmax the tiebreaker. Row 215 (age 29, tauxmax 202) remains first; among the age-34 patients, the one with lower tauxmax appears before the other.

Step 50: Histogram of tauxmax

```
hist(m$tauxmax)
```

```
> hist(m$tauxmax)
```

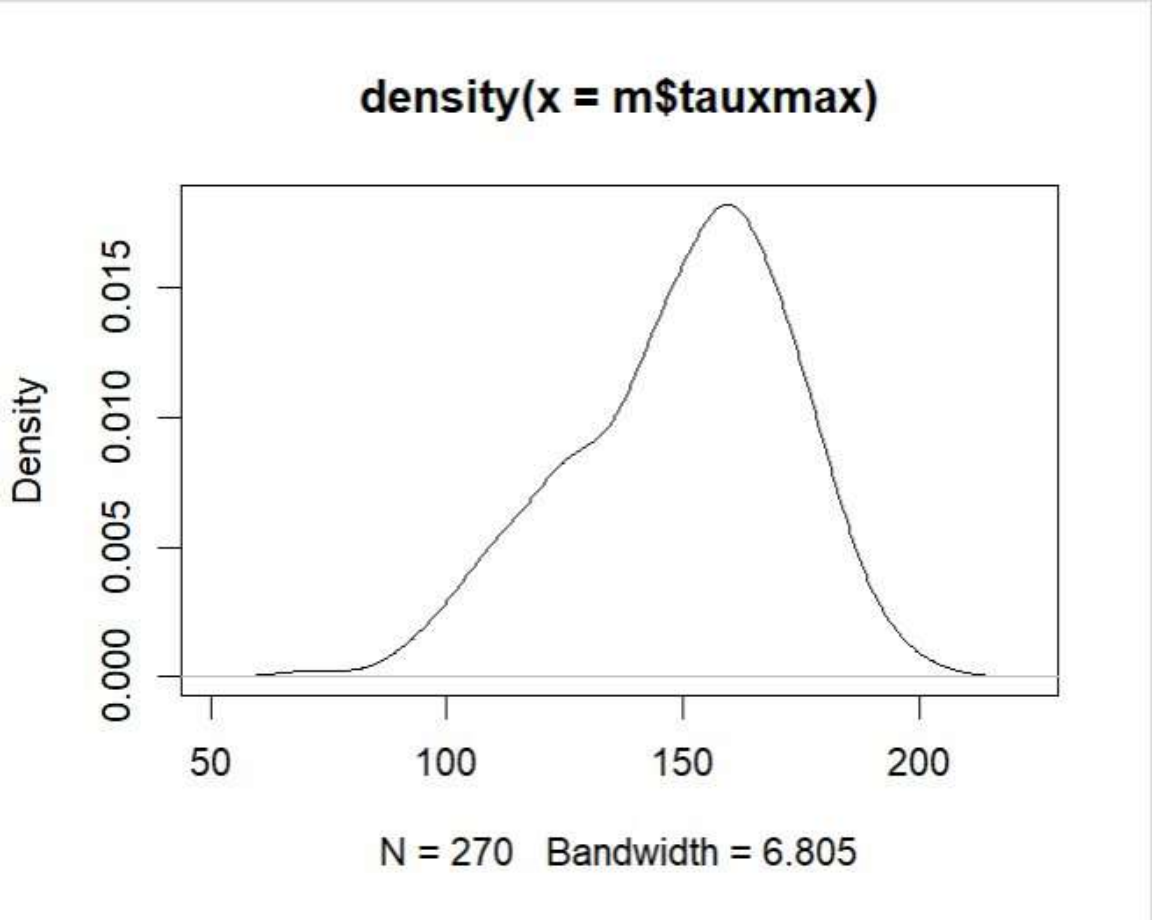


hist() generates a frequency histogram of maximum heart rate. The distribution is unimodal and roughly bell-shaped, centred around 150–160 bpm, with a slight left skew. Very few patients fall below 100 bpm or above 190 bpm.

Step 51: Density Plot of tauxmax

```
plot(density(m$tauxmax))
```

```
> plot(density(m$tauxmax))
```



density() computes a kernel density estimate (bandwidth = 6.805, N = 270). The smooth curve confirms the unimodal shape seen in the histogram, peaking near 155 bpm and tapering gradually toward both extremes.

Step 52: Installing and Loading the sm Package

```
install.packages("sm")
library(sm)
```

```
> install.packages("sm")
WARNING: Rtools is required to build R packages but is not currently installed. Please download and install th
e appropriate version of Rtools before proceeding:

https://cran.rstudio.com/bin/windows/Rtools/
Installing package into 'C:/Users/Younes/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/sm_2.2-6.0.zip'
Content type 'application/zip' length 1018144 bytes (994 KB)
downloaded 994 KB

package 'sm' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
        C:\Users\Younes\AppData\Local\Temp\RtmpkjRm4N\downloaded_packages
```

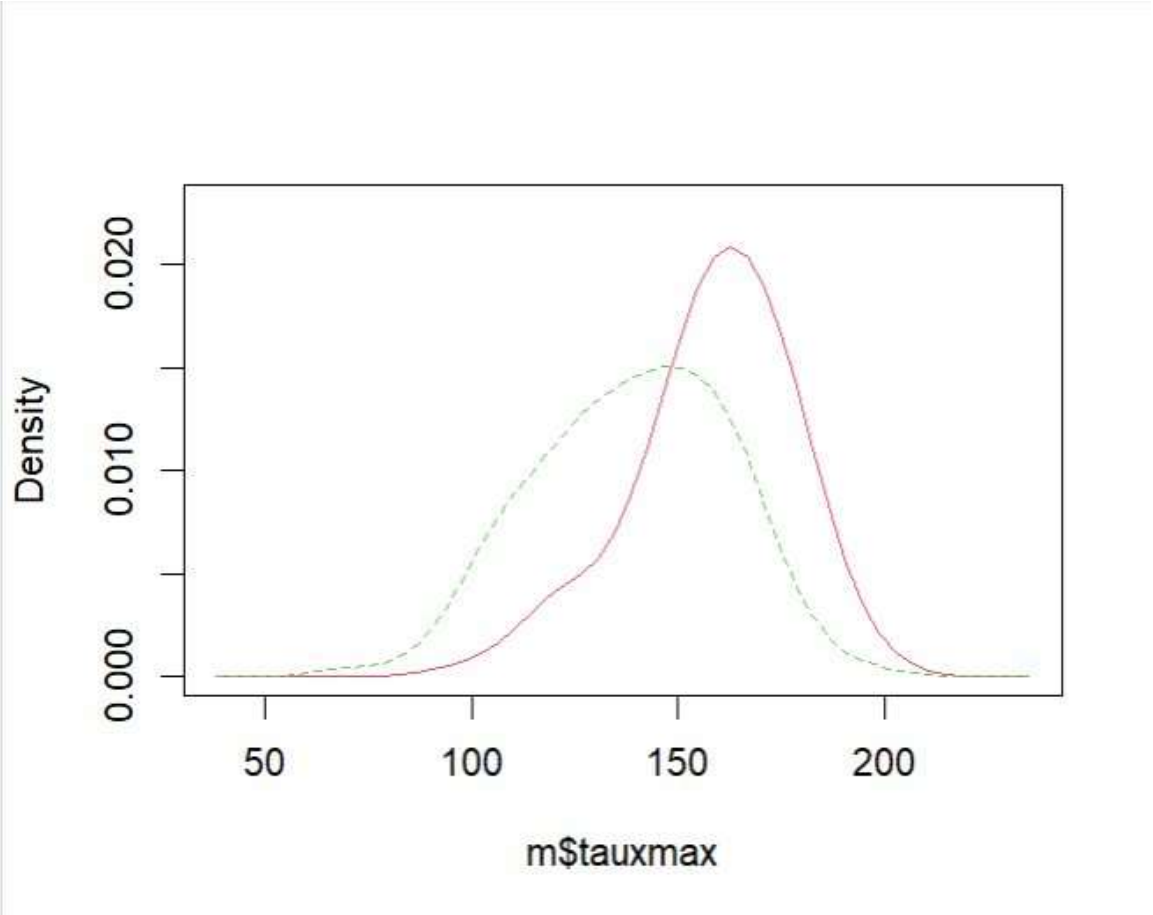
```
> library(sm)
Package 'sm', version 2.2-6.0: type help(sm) for summary information
warning message:
package 'sm' was built under R version 4.5.3
```

The sm package (version 2.2-6.0) provides sm.density.compare() for overlaid group-specific density curves. It is fetched from CRAN, unpacked, and loaded successfully; a minor version warning is non-critical and does not affect functionality.

Step 53: Comparative Density: tauxmax by coeur

```
sm.density.compare(m$tauxmax, m$coeur)
```

```
> sm.density.compare(m$tauxmax,m$coeur)
```

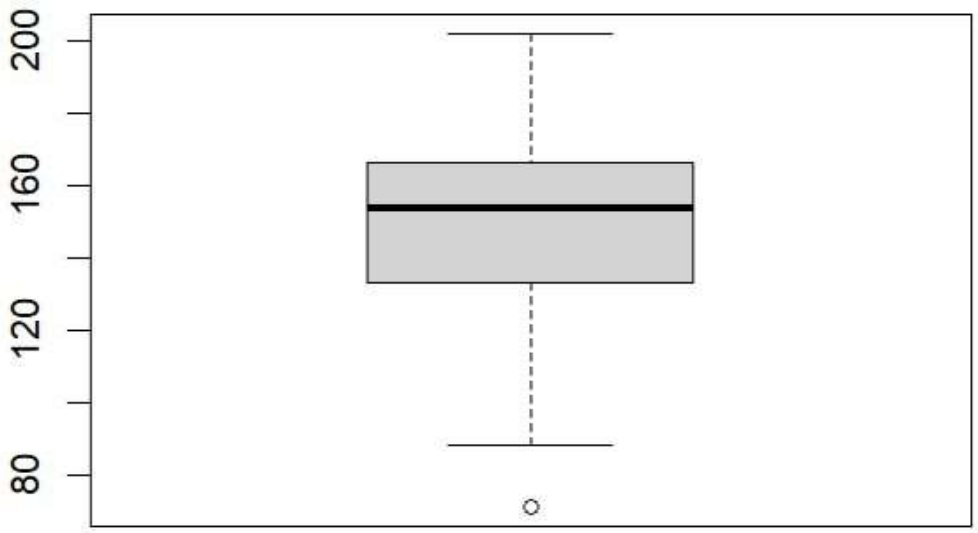


sm.density.compare() overlays two kernel density curves — one for each level of coeur. The 'absence' group (solid pink) peaks higher and further right (~155–160 bpm), while 'presence' (dashed green) is broader and shifted left, indicating lower tauxmax in diseased patients.

Step 54: Boxplot of tauxmax

```
boxplot(m$tauxmax)
```

```
> boxplot(m$tauxmax)
```

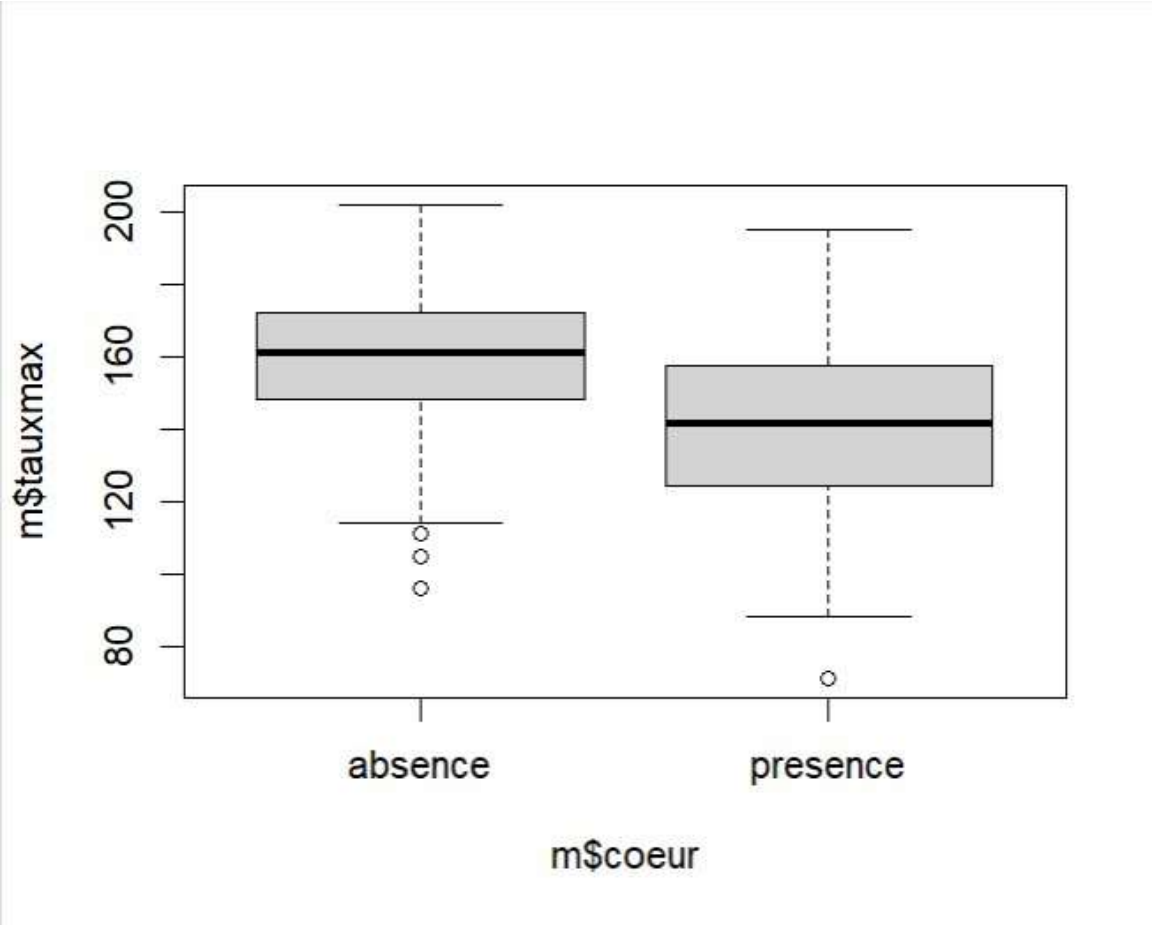


A single boxplot displays the five-number summary: median ≈ 153.5, IQR from 133 to 166, whiskers extending to approximately 88 and 200, with one outlier below 80. The compact IQR relative to the full range indicates moderate variability.

Step 55: Grouped Boxplot: tauxmax by coeur

```
boxplot(m$tauxmax ~ m$coeur)
```

```
> boxplot(m$tauxmax ~ m$coeur)
```

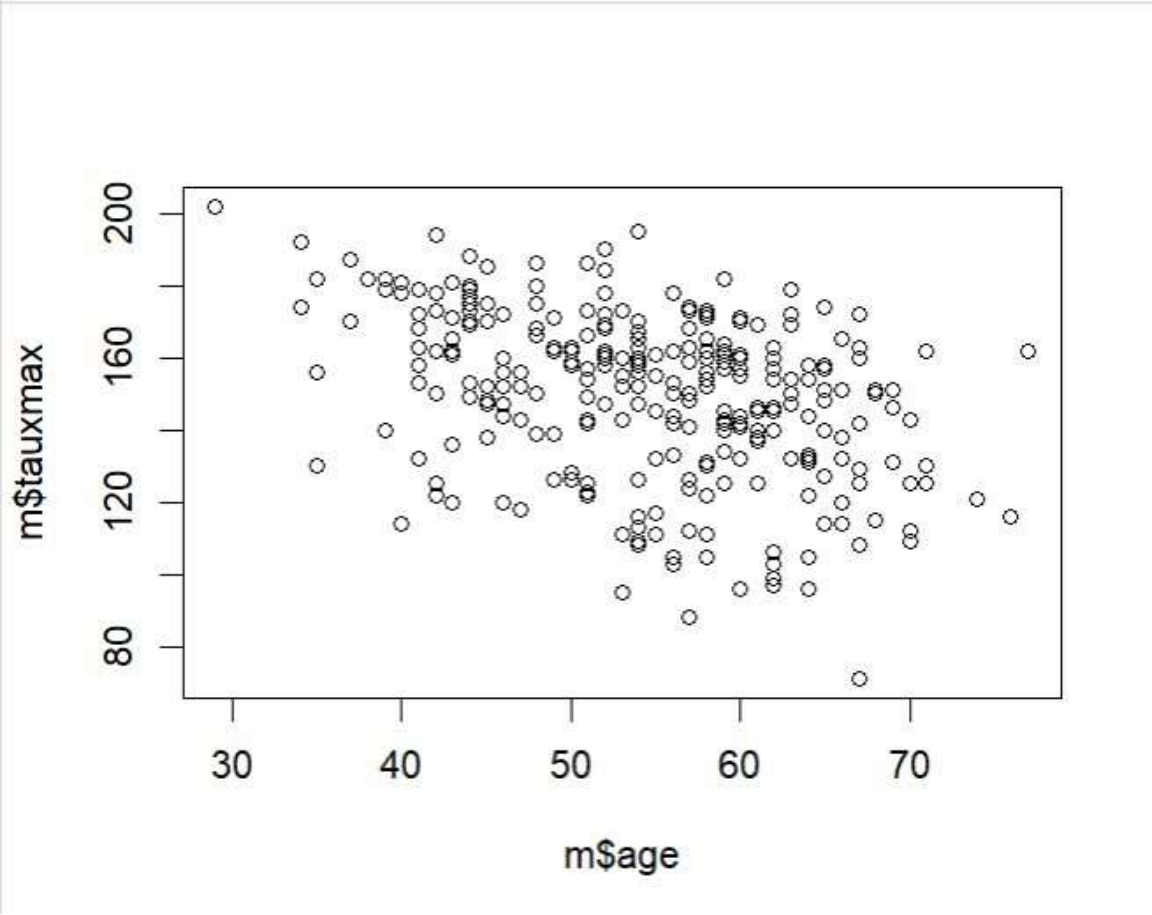


The tilde formula splits the boxplot by heart disease status. The 'absence' box is notably higher (median ≈ 160 bpm) than 'presence' (median ≈ 130 bpm), with both outliers appearing in the 'presence' group. This visual separation corroborates the density comparison.

Step 56: Scatter Plot: Age vs tauxmax

```
plot(m$age, m$tauxmax)
```

```
> plot(m$age,m$tauxmax)
```



The 2D scatter plot reveals a mild negative trend: as age increases, maximum heart rate tends to decline. The cloud of points is wide, indicating high individual variability, but the downward slope aligns with the known physiological reduction of maximum HR with age.

Step 57: Coloured Scatter Plot by Heart Disease

```
plot(m$age, m$tauxmax, pch=21, bg=c("green","red")[unclass(m$coeur)])
```

```
> plot(m$age,m$tauxmax,pch=21,bg=c("green","red")[unclass(m$coeur)])
```

Adding pch=21 (filled circles) and colour-coding by coeur distinguishes healthy (green) from diseased (red) patients. Green points cluster at higher tauxmax values for all ages, while red points concentrate in the lower-right quadrant (older, lower tauxmax).

Step 58: Listing Column Names

```
colnames(m)
```

```
> colnames(m)
[1] "age"        "sexe"       "typedouleur" "sucre"      "tauxmax"    "angine"     "depression"
[8] "coeur"
```

colnames() lists the 8 variable names: age, sexe, typedouleur, sucre, tauxmax, angine, depression, coeur. This inventory step precedes the creation of a derived variable and confirms the column order needed for cbind().

Step 59: Creating the tauxnet Variable

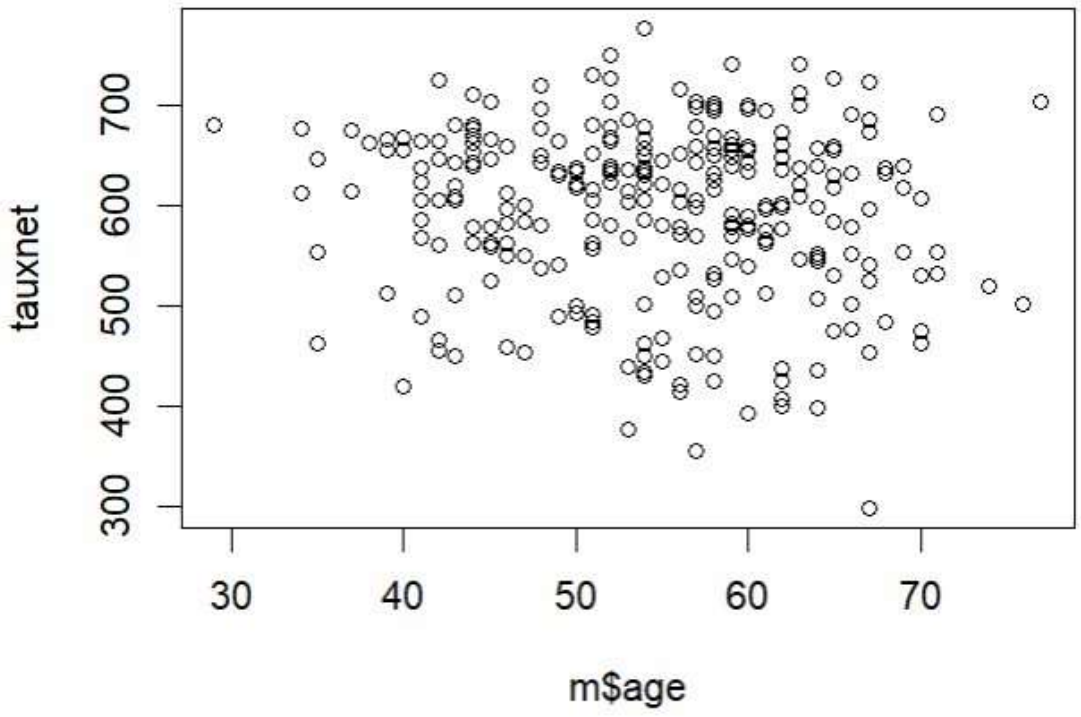
```
tauxnet <- m$taux * log(m$age)
plot(m$age, tauxnet, pch=21, bg=c('green','red')[unclass(m$coeur)])
```

```
> plot(m$age,m$tauxmax,pch=21,bg=c("green","red")[unclass(m$coeur)])
```

```
> tauxnet <- m$taux*log(m$age)
```

```
  tauxnet          num [1:270] 463 673 570 437 521 ...
```

```
> plot(m$age,tauxnet,pch=21,bg=c("green","red")[unclass(m$coeur)])
```



tauxnet is defined as m$taux × log(m$age), producing a log-scaled rate variable. The environment panel confirms 270 numeric values. The scatter plot of age vs tauxnet shows a flatter, denser cloud than the raw tauxmax plot — the log transformation compresses age-related variance while preserving the green/red class separation.

Step 60: Adding tauxnet to the Data Frame

```
m <- cbind(m, tauxnet)
colnames(m)
```

```
> m <- cbind(m,tauxnet)
```

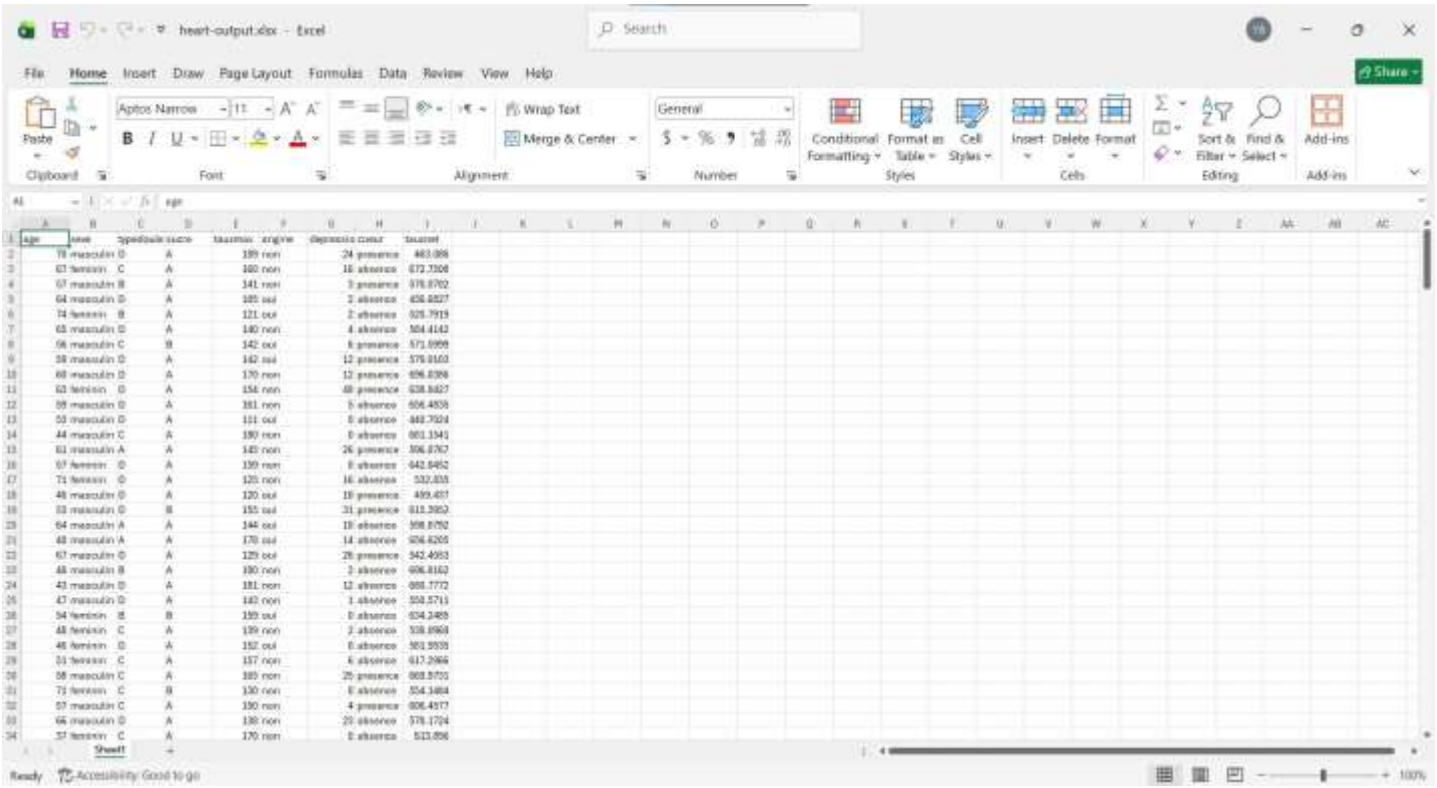
```
> colnames(m)
[1] "age"          "sexe"         "typedouleur" "sucre"        "tauxmax"      "angine"       "depression"
[8] "coeur"        "tauxnet"
```

cbind() appends tauxnet as a ninth column to m. colnames() confirms the updated schema: the original 8 variables followed by 'tauxnet'. The dataset is now enriched with the derived feature, ready for export.

Step 61: Exporting the Enriched Dataset to Excel

```
write.xlsx(m, file="heart-output.xlsx", row.names=F)
```

```
> write.xlsx(m,file="heart-output.xlsx",row.names=F)
```



write.xlsx() serialises the 270-row, 9-column data frame to heart-output.xlsx without row index numbers. The Excel screenshot confirms all columns are present — including tauxnet values such as 463.09 and 672.75 — verifying a complete and accurate export of the processed dataset.

Conclusion

This lab provided a comprehensive walkthrough of data manipulation in R using the heart disease dataset. Key competencies exercised include: installing and using the xlsx package for Excel I/O, multi-dimensional data frame indexing, Boolean and multi-criteria filtering, descriptive statistics and cross-tabulations with tapply() and table(), a full suite of univariate and bivariate graphical methods, creation of derived variables, and exporting enriched data. Together these form the foundational data-handling toolkit required for machine learning preprocessing in subsequent TPs.